

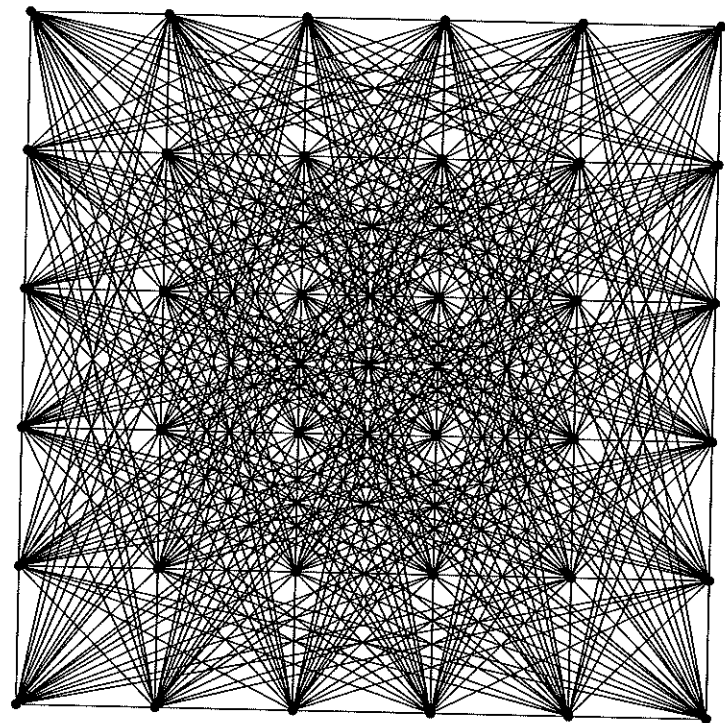
Graph Theory and Complex Networks

An Introduction

Maarten van Steen

CHAPTER 2

FOUNDATIONS



In the previous chapter we have informally introduced the notion of a network and have given several examples. In order to study networks, we need to use a terminology that allows us to be precise. For example, when we speak about the distance between two nodes in a network, what do we really mean? Likewise, is it possible to specify how well connected a network is? These and other statements can be formulated accurately by adopting terminology from **graph theory**. Graph theory is a field in mathematics that gained popularity in the 19th and 20th century, mainly because it allowed to describe phenomena from very different fields: communication infrastructures, drawing and coloring maps, scheduling tasks, and social structures, just to name a few.

We will first concentrate only on the foundations of graph theory. To this end, we will use the language of mathematics, as it allows us to be precise and concise. However, to many this language with its many symbols and often peculiar notations can easily form an obstacle to grasp the essence for what it is being used. For this reason, we will gently and gradually introduce notations while providing more verbose descriptions alongside the more formal definitions. You are encouraged to pay explicit attention to the formalities: in the end, they will prove to be much more convenient to use than verbose verbal descriptions. The latter often simply fail to be precise enough to completely understand what is going on. It is also not that difficult, as most notations come directly from set theory.

2.1 Formalities

Let us start with discussing what is actually meant by a network. To this end, we first concentrate on some basic formal concepts and notations from graph theory, together with a few fundamental properties that characterize networks. After having studied this section, you will have already learned a lot about the world of graphs and should also feel more comfortable with mathematical notations.

Graphs and vertex degrees

As said, the networks that have been introduced so far are mathematically known as graphs. In its simplest form, a graph is a collection of vertices that can be connected to each other by means of edges. In particular, each edge of graph joins exactly two vertices. Using a formal notation, a graph is defined as follows.

Definition 2.1: A graph G consists of a collection V of vertices and a collection of edges E , for which we write $G = (V, E)$. Each edge $e \in E$ is said to join two

vertices, which are called its **end points**. If e joins $u, v \in V$, we write $e = \langle u, v \rangle$. Vertex u and v in this case are said to be **adjacent**. Edge e is said to be **incident** with vertices u and v , respectively.

We will often write $V(G)$ and $E(G)$ to denote the set of vertices and edges associated with graph G , respectively. It is important to realize that an edge can actually be represented as an *unordered* tuple of two vertices, that is, its end points. For this reason, we make no distinction between $\langle v, u \rangle$ and $\langle u, v \rangle$: they both represent the fact that vertex u and v are adjacent.

This definition may already raise a few questions. First of all, is it possible that an edge joins the same vertices, that is, can an edge form a **loop**? There is nothing in the definition that prevents this, and indeed, such edges are allowed. Likewise, you may be wondering whether two vertices u and v may be joined by **multiple edges**, that is, a set of edges each having u and v as their end points. Indeed, this is also possible, and we shall be discussing a few examples shortly. A graph that does not have loops or multiple edges is called **simple**.

Likewise, there is nothing that prohibits a graph from having no vertices at all. Of course, in that case there will also be no edges. Such a trivial graph is called **empty**. Another special case is formed by a simple graph having n vertices, with each vertex being adjacent to every other vertex. This graph is also known as a **complete graph**. A complete graph with n vertices is commonly denoted as K_n . Finally, the **complement** of a graph G , denoted as $\bar{G}$ is the graph obtained from G by removing all its edges and joining exactly those vertices that were *not* adjacent in G . It should be clear that if we take a graph G and its complement $\bar{G}$ “together,” we obtain a complete graph. Taking two graphs “together” will be made more precise later in this chapter.

As an aside, notice that when we write $\langle u, v \rangle$, we can say only that u and v are adjacent, that is, that there is *at least one* edge that joins the two. Strictly speaking, it is not possible using this notation to distinguish different edges that all happen to join both u and v . If we wanted to make that distinction, we would have to write something like $e_1 = \langle u, v \rangle$ and $e_2 = \langle u, v \rangle$. In other words, we would have to explicitly enumerate the edges that join u and v . Of course, when dealing with *simple* graphs, there can be no mistake about which edge we are considering when we write $\langle u, v \rangle$. Here we see an example where mathematics allows us to be precise and unambiguous. We will encounter many more of such examples.

As in so many practical situations, it is often convenient to talk about your neighbors. In graph-theoretical terms, the neighbors of a vertex u are formed by the vertices that are adjacent to u , or, in other words, those ver-

tices to which v has been joined by means of an edge. We can formulate this precisely using formal mathematical notations as follows.

Definition 2.2: For any graph G and vertex $v \in V(G)$, the **neighbor set** $N(v)$ of v is the set of vertices (other than v) adjacent to v , that is

$$N(v) \stackrel{\text{def}}{=} \{w \in V(G) \mid v \neq w, \exists e \in E(G) : e = \langle u, v \rangle\}$$

Note 2.1 (Mathematical language)

The formal notation in Definition 2.2 is very precise, yet can be somewhat intimidating. Let us decypher it a bit. First, we use the symbol $\stackrel{\text{def}}{=}$ to express that what is written on the left-hand side is defined by what is written on the right-hand side. In other words,

$$N(v) \stackrel{\text{def}}{=} \dots$$

is nothing but accurately stating that $N(v)$ is defined by what follows on the right hand of $\stackrel{\text{def}}{=}$. Recall that the symbol ' $\exists$ ' is the **existential quantifier** used in set theory to express statements like "there exists an ..." Keeping this in mind, you should now be able to see that the right-hand side translates into English to the following statement:

The set of vertices w in G , with w not equal to v , such that there exists an edge e in G that joins v and w .

We will be encountering many more of these formal statements. If you have trouble correctly interpreting them, we encourage you to make translations like the previous one to actually practice *reading* mathematics. After a while, you will notice that these translations come naturally by themselves.

The word "graph" comes from the fact that it is often very convenient to use a graphical representation, as shown in Figure 2.1. In this example, we have a graph G with eight vertices and a total of 18 edges. Each vertex is represented as a black dot whereas edges are drawn as lines. When drawing a graph, it is often convenient to add labels. Both vertices and edges can be labeled. We shall generally not use subscripts when labeling vertices and edges in our drawings of graphs. This means that a label such as e_{13} from Figure 2.1 is the same as e_{13} in our text.

It should be clear that there may be many different ways to draw a graph. In the first place, there is no reason why we would stick to just dots and lines, although it is common practice to do so. Secondly, there are, in principle, no rules concerning on where to position the drawn vertices, nor are there any rules stating that a line should be drawn in a straight fashion. However, the way that we draw graphs is often important when it comes to visualizing certain aspects. We return to this issue extensively in Section 2.4.

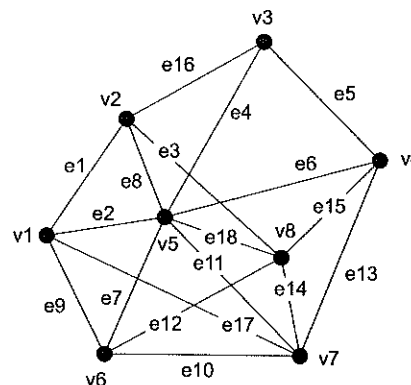


Figure 2.1: An example of a graph with eight vertices and 18 edges.

$$\begin{aligned} V(G) &= \{v_1, \dots, v_8\} \\ E(G) &= \{e_1, \dots, e_{18}\} \\ e_1 &= \langle v_1, v_2 \rangle & e_{10} &= \langle v_6, v_7 \rangle \\ e_2 &= \langle v_1, v_5 \rangle & e_{11} &= \langle v_5, v_7 \rangle \\ e_3 &= \langle v_2, v_8 \rangle & e_{12} &= \langle v_6, v_8 \rangle \\ e_4 &= \langle v_3, v_5 \rangle & e_{13} &= \langle v_4, v_7 \rangle \\ e_5 &= \langle v_3, v_4 \rangle & e_{14} &= \langle v_7, v_8 \rangle \\ e_6 &= \langle v_4, v_5 \rangle & e_{15} &= \langle v_4, v_8 \rangle \\ e_7 &= \langle v_5, v_6 \rangle & e_{16} &= \langle v_2, v_3 \rangle \\ e_8 &= \langle v_2, v_5 \rangle & e_{17} &= \langle v_1, v_7 \rangle \\ e_9 &= \langle v_1, v_6 \rangle & e_{18} &= \langle v_5, v_8 \rangle \end{aligned}$$

An important property of a vertex is the number of edges that are incident with it. This number is called the **degree** of a vertex.

Definition 2.3: The number of edges incident with a vertex v is called the **degree** of v , denoted as $\delta(v)$. Loops are counted twice.

Let us consider our example from Figure 2.1 again. In this case, because there are four edges incident with vertex v_1 , we have that $\delta(v_1) = 4$. We can complete the picture by considering every vertex, which gives us:

Vertex	Degree	Incident edges	Neighbors
v_1	4	$\langle v_1, v_2 \rangle, \langle v_1, v_5 \rangle, \langle v_1, v_6 \rangle, \langle v_1, v_7 \rangle$	v_2, v_5, v_6, v_7
v_2	4	$\langle v_1, v_2 \rangle, \langle v_2, v_3 \rangle, \langle v_2, v_5 \rangle, \langle v_2, v_8 \rangle$	v_1, v_3, v_5, v_8
v_3	3	$\langle v_2, v_3 \rangle, \langle v_3, v_4 \rangle, \langle v_3, v_5 \rangle$	v_2, v_4, v_5
v_4	4	$\langle v_3, v_4 \rangle, \langle v_4, v_5 \rangle, \langle v_4, v_7 \rangle, \langle v_4, v_8 \rangle$	v_3, v_5, v_7, v_8
v_5	7	$\langle v_1, v_5 \rangle, \langle v_2, v_5 \rangle, \langle v_3, v_5 \rangle, \langle v_4, v_5 \rangle, \langle v_5, v_6 \rangle, \langle v_5, v_7 \rangle, \langle v_5, v_8 \rangle$	$v_1, v_2, v_3, v_4, v_6, v_7, v_8$
v_6	4	$\langle v_1, v_6 \rangle, \langle v_5, v_6 \rangle, \langle v_6, v_7 \rangle, \langle v_6, v_8 \rangle$	v_1, v_5, v_7, v_8
v_7	5	$\langle v_1, v_7 \rangle, \langle v_4, v_7 \rangle, \langle v_5, v_7 \rangle, \langle v_6, v_7 \rangle, \langle v_7, v_8 \rangle$	v_1, v_4, v_5, v_6, v_8
v_8	5	$\langle v_2, v_8 \rangle, \langle v_4, v_8 \rangle, \langle v_5, v_8 \rangle, \langle v_6, v_8 \rangle, \langle v_7, v_8 \rangle$	v_2, v_4, v_5, v_6, v_7

When adding the degrees of all vertices from G , we find that the total sum is 36, which is exactly twice the number of edges. This brings us to our first theorem:

Theorem 2.1: For all graphs G , the sum of the vertex degrees is twice the number of edges, that is,

$$\sum_{v \in V(G)} \delta(v) = 2 \cdot |E(G)|$$

Proof. When we count the edges of a graph G by enumerating for each vertex v of G the edges incident with that vertex v , we are counting each edge exactly twice. Hence, $\sum_{v \in G} \delta(v) = 2 \cdot |E(G)|$. $\square$

Note 2.2 (Mathematical language)

Again, we encounter some formal mathematical notations. In this case, we use the standard symbol $\sum$ as an abbreviation for summation. Thus, $\sum_{i=1}^n x_i$ is the same as $x_1 + x_2 + x_3 + \dots + x_n$. In many cases, the summation is simply over all elements in a specific set, such as in our example where we consider all the vertices in a graph. In that case, if we assume that $V(G)$ consists of the vertices $v_1, v_2, \dots, v_n$, the notation $\sum_{v \in V(G)} \delta(v)$ is to be interpreted as:

$$\sum_{v \in V(G)} \delta(v) \stackrel{\text{def}}{=} \delta(v_1) + \delta(v_2) + \dots + \delta(v_n)$$

Note, furthermore, that we use the notation $|S|$ to denote the size of a set S . In our example, $|E(G)|$ thus denotes the size of $E(G)$ or, in other words, the total number of edges in graph G .

There is also an interesting corollary that follows from this property, namely that the number of vertices with an odd degree must be even. This can be easily seen if we split the vertices V of a graph into two groups: V_{odd} containing all vertices with odd degree, and V_{even} with all vertices having even degree. Clearly, if we take the sum of all the degrees from vertices in V_{odd} , and those from V_{even} , we will have summed up all vertex degrees, that is,

$$\sum_{v \in V_{\text{odd}}} \delta(v) + \sum_{v \in V_{\text{even}}} \delta(v) = \sum_{v \in V} \delta(v)$$

which is even. Because the sum of even vertex degrees is obviously even, we know that $\sum_{v \in V_{\text{even}}} \delta(v)$ is even. This can only mean that $\sum_{v \in V_{\text{odd}}} \delta(v)$ must also be even. Combining this with the fact that all vertex degrees in V_{odd} are odd, we conclude that the number of vertices with odd degree must be even, that is, $|V_{\text{odd}}|$ is even. We have thus just proven:

Corollary 2.1: *For any graph, the number of vertices with odd degree is even.*

The vertex degree is a simple, yet powerful concept. As we shall see throughout this text, vertex degrees are used in many different ways. For example, when considering social networks, we can use vertex degrees to express the importance of a person within a social group. Also, when we discuss the structure of real-world communication networks such as the Internet, it will turn out that we can learn a lot by considering the *distribution* of vertex degrees. More specifically, by simply ordering vertices by their

vertex degree, we will be able to obtain insight in how such a network is actually organized.

Degree sequence

Listing the vertex degrees of a graph gives us a **degree sequence**. The vertex degrees are usually listed in descending order, in which case we refer to an **ordered degree sequence**. For example, if we consider the eight vertices of graph G from Figure 2.1, we have the following vertex degrees

vertex:	v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8
degree:	4	4	3	4	7	4	5	5

which, when ordering these degrees in descending order, leads to the ordered degree sequence

$$[7, 5, 5, 4, 4, 4, 4, 3]$$

If every vertex has the same degree, the graph is called **regular**. In a **k-regular** graph each vertex has degree k . As a special case, 3-regular graphs are also called **cubic graphs**.

When considering degree sequences, it is common practice to focus only on simple graphs, that is, graphs without loops and multiple edges. An interesting question that comes to mind is when we are given a list of numbers, is there also a simple graph whose degree sequence corresponds to that list? There are some obvious cases where we already know that a given list cannot correspond to a degree sequence. For example, we have just proven that the sum of vertex degrees is always even. Therefore, a minimal requirement is that the sum of the elements of that list should be even as well. Likewise, it is not difficult to see that, for example, the sequence $[4, 4, 3, 3]$ cannot correspond to a degree sequence. In this case, if this were a degree sequence, we would be dealing with a graph of four vertices. The first vertex is supposed to have four incident edges. In the case of simple graphs, each of these edges should be incident with a different vertex. However, there are only three vertices left to choose from, so $[4, 4, 3, 3]$ can never correspond to the degree sequence of a simple graph.

Of course, taking a trial-and-error approach to see whether a list corresponds to a degree sequence is not the way to go. Fortunately, there is a systematic way to see whether a given list of numbers corresponds to the degree sequence of a simple graph, in which case the sequence is said to be **graphic**. Let's return to our graph from Figure 2.1, but now assume that we are given only the list $[7, 5, 5, 4, 4, 4, 4, 3]$. We ask ourselves whether this list is graphic. If this is the case, we should be able to construct a graph that has this degree sequence. Note that this graph need not necessarily be the same as the one from Figure 2.1. This is how we can address this issue.

- Consider $[7, 5, 5, 4, 4, 4, 3]$. If this sequence is graphic corresponding to a graph, say G_1 , then we should be able to construct G_1 from another graph G_2 by adding a vertex v_1 to G_2 and joining v_1 to seven other vertices from G_2 . This would then explain that G_1 has a vertex with highest degree 7. Note that for this construction to work, it is necessary that we can construct G_2 .

It should be clear that if we do not change the ordering of vertex degrees, that the degree sequence of G_2 is equal to $[4, 4, 3, 3, 3, 3, 2]$. First, it contains one element less than the degree sequence of G_1 . Second, the first element of the degree sequence of G_2 corresponds to the second element of G_1 's degree sequence: it's the degree of the same vertex, yet for G_2 it should be one less than in G_1 because this vertex is not yet joined to the added vertex v_1 . Likewise, the second element of G_2 's degree sequence corresponds to the third one in the degree sequence of G_1 , and so on.

- If $[4, 4, 3, 3, 3, 2]$ is graphic we can apply the same trick: G_2 should be constructable from a graph G_3 by adding a vertex v_2 and joining v_2 to four vertices from G_3 . Following a completely analogous procedure as before, v_2 is joined to the vertices from G_3 such that these vertices will then have vertex degree 4, 3, 3, and 3, respectively. This can only mean that in G_3 they will have degree 3, 2, 2, and 2, respectively, leading to the following list: $[3, 2, 2, 2, 3, 2]$.

Note that in this example, the fifth element is the same as the sixth element in the degree sequence of G_2 . The first four elements represent vertices that will be joined to the new vertex v_2 . The other elements represent vertices that remain untouched, and will thus have the same number of incident edges in G_2 .

- Continuing this line of reasoning, if $[3, 3, 2, 2, 2, 2]$ is the (now ordered) degree sequence of G_3 , then we should be able to construct G_3 from a graph G_4 to which we have added a vertex v_3 . This vertex would be joined to the vertices having degree 2, 1, and 1 in G_4 , respectively, yielding the list $[2, 1, 1, 2, 2]$. Again, note that this list contains one element less than the degree sequence of G_3 , but that now its fourth and subsequent elements represent vertices that have the same vertex degree in G_4 and G_3 .
- We now have that if ordered list $[2, 2, 2, 1, 1]$ is graphic, then so should $[1, 1, 1, 1]$, corresponding to a graph G_5 .
- Likewise, if $[1, 1, 1, 1]$ is graphic, then so should the list of vertex degrees $[0, 1, 1]$ correspond to a graph G_6 .
- Finally, if the ordered list $[1, 1, 0]$ is graphic, then so should $[0, 0]$,

which is true: it is a graph G_7 with two vertices and no edges.

We can safely conclude that the sequence $[7, 5, 5, 4, 4, 4, 3]$ indeed corresponds to a simple graph. The construction of the graph G_1 is illustrated in Figure 2.2 which shows how each graph $G_1, G_2, \dots, G_6$ is constructed by adding a vertex to the previous one, starting from graph G_7 . The answer to whether G_1 is the same as the graph from Figure 2.1 is a question we defer until later. In fact, it turns out to be question that is generally not easy to resolve.

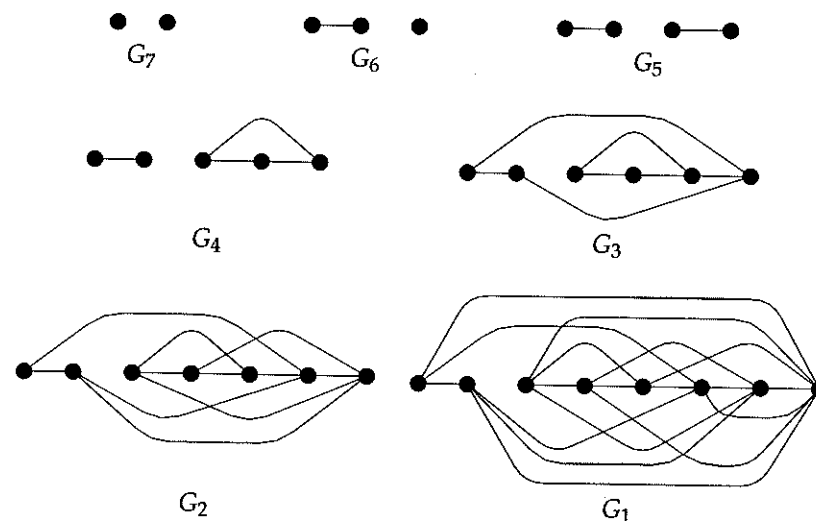


Figure 2.2: The construction of graph G_1 from previous graphs based on degree sequences.

Intuitively, it should be clear that we have just introduced a *systematic* way of checking whether a given list of numbers corresponds to the degree sequence of a graph. It also forms the essence of the proof of the following theorem that tells us when a list of numbers is indeed graphic.

Theorem 2.2 (Havel-Hakimi): Consider a list $s = [d_1, d_2, \dots, d_n]$ of n numbers in descending order. This list is graphic if and only if $s^* = [d_1^*, d_2^*, \dots, d_{n-1}^*]$ of $n - 1$ numbers is graphic as well, where

$$d_i^* = \begin{cases} d_{i+1} - 1 & \text{for } i = 1, 2, \dots, d_1 \\ d_{i+1} & \text{otherwise} \end{cases}$$

Note 2.3 (Mathematical language)

Note that this theorem consists of *two* statements:

1. if s^* is graphic then so is s
2. if s is graphic then so is s^*

This is the meaning of “if and only if,” which is often abbreviated to *iff*. We will encounter more of such theorems, and in order to prove them correct, proofs in these cases will always consist of two parts.

Proof of Theorem 2.2. To prove this theorem, let us first assume that s^* is graphic. We then need to show that s is also graphic. Let G^* be a simple graph with degree sequence s^* . We now construct a simple graph G from G^* with degree sequence s as follows (and in doing so, we show that s is graphic). Take G^* and add a vertex u . For readability, let $k = d_1$ and consider the k vertices $v_1, v_2, \dots, v_k$ from G^* having respectively degree $d_1^*, d_2^*, \dots, d_k^*$. We then join these vertices to the newly added vertex u . Obviously, u now has degree k , but also each vertex v_i now has degree $d_i^* + 1$. Because all other vertices of G^* are not joined with u , their vertex degree is left unaffected. As a consequence, the newly constructed graph G has degree sequence $[k, d_1^* + 1, d_2^* + 1, \dots, d_k^* + 1, d_{k+1}^*, \dots, d_{n-1}^*]$, which is precisely s .

Let us now consider the opposite: if s is graphic, we need to show that s^* is so as well. In other words, we need to find a graph G^* that has degree sequence s^* . To this end, we consider three different sets of vertices from G . Let u be a vertex with degree $k = d_1$. Let $V = \{v_1, v_2, \dots, v_k\}$ be the respective vertices with the k next highest degrees $d_2, d_3, \dots, d_{k+1}$. Finally, let $W = \{w_1, w_2, \dots, w_{n-k-1}\}$ be the remaining $n - k - 1$ vertices with degree $d_{k+2}, d_{k+3}, \dots, d_n$, respectively.

Consider the graph G^* by removing u from G , along with the k edges incident with u . If each of these edges is incident with one of the vertices from V , then obviously G^* is a graph with degree sequence $(d_2 - 1, d_3 - 1, \dots, d_{k+1} - 1, d_{k+2}, \dots, d_n)$, which is precisely s^* .

Now consider the situation that u is adjacent to a vertex from W , say w_i . If for some vertex $v_j \in V$, the degree of v_j and w_i are the same, i.e., $\delta(w_i) = \delta(v_j)$, then we can simply swap w_i and v_j in the original construction of the sets V and W , meaning that $\langle u, w_i \rangle$ is now an edge incident with a vertex from V instead of W . However, if $\delta(w_i) < \delta(v_j)$ (i.e., $\delta(w_i)$ is less than the degree of any vertex from V) we cannot apply such an exchange.

The problem that we need to solve is that there is now a vertex v_j not adjacent to u whose degree will remain the same instead of being decremented by 1. Likewise, by simply removing u we would decrease the degree of w_i ,

while we would like to see it unaffected if we want to realize the degree sequence s^* . Note, however, that because $\delta(v_j) > \delta(w_i)$, there is a vertex x adjacent to v_j but not adjacent to w_i (note also that $x \neq u$), as shown in Figure 2.3(a). In constructing G^* we now first remove edges $\langle u, w_i \rangle$ and $\langle v_j, x \rangle$, and then add edges $\langle x, w_i \rangle$ and $\langle u, v_j \rangle$, leading to the situation shown in Figure 2.3(b). The effect is that we now have a graph G' in which u is adjacent to v_j instead of w_i , but without affecting the degree of u, v_j, x , or w_i . In other words, G' has the degree sequence s . If u is now adjacent to vertices only from V , we have already shown that s^* is graphic. If u is still adjacent to a vertex from W , we apply the same method to construct a graph G'' in which u is adjacent to one more vertex from V . If necessary, we repeat this method until u is adjacent only to vertices from V , at which point we know that s^* is graphic. $\square$

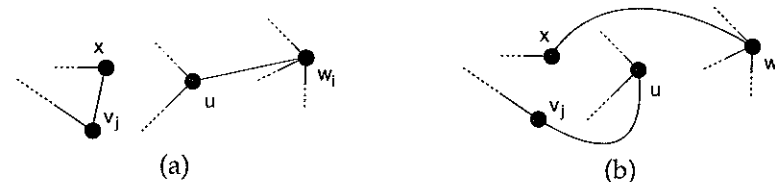


Figure 2.3: Changing a graph so that it meets the sets V and W of the Havel-Hakimi proof.

Note 2.4 (Proof techniques)

The proof of the Havel-Hakimi theorem illustrates a number of important issues in graph theory. In the first place, it is a **proof by construction**. In the case of the Havel-Hakimi theorem this means that we show that the theorem holds by actually constructing a graph from a given degree sequence. In general, proving properties by construction is very powerful: not only do we demonstrate the *existence* of a property, we also show how to get there. In contrast, with *non-constructive proofs* we merely prove that some property must exist, often by first assuming that it does not exist and subsequently arriving at a contradiction. We will come across more of these proofs, but also ones in which we merely show that a property must exist, without giving a graph that has the specific property.

Another important issue in proving the Havel-Hakimi theorem, is that we show the power of visualization. Visualizing situations, either explicitly on paper or otherwise merely in your mind, is particularly useful in the case of graphs, and should come as no surprise. When graphs are studied for the first time, it is tempting to draw complete examples, that is, graphs in which each

edge joins two vertices. However, as you become more experienced, it turns out that sketching graphs as is done in Figure 2.3 is actually more illustrative as these drawings reflect the essence of what you are trying to prove. Irrelevant details are thus avoided. You are encouraged to go for the sketches.

Note that two graphs with the same degree sequence need not be the same. In other words, when given a degree sequence, it may be possible to construct several, different, graphs that have that sequence, as is illustrated in Figure 2.4. The two graphs in Figure 2.4(a) have the same degree sequence, yet they are truly different. The same holds for the two graphs from Figure 2.4(b). We return to the notion of similarity of graphs in Section 2.2.

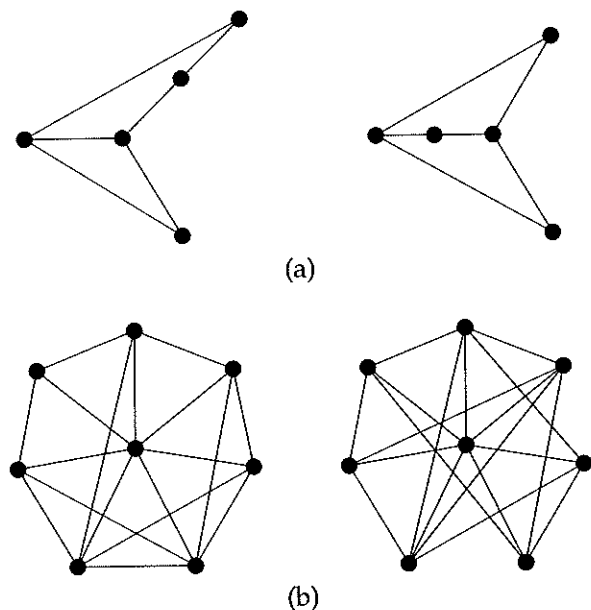


Figure 2.4: Different graphs with the same ordered degree sequence: (a) $[3, 3, 2, 2, 2, 2]$, and (b) $[7, 5, 5, 4, 4, 4, 4, 3]$.

Subgraphs and line graphs

Another important concept of graphs is that of a subgraph. A graph H is a subgraph of G if H consists of a subset of the edges and vertices of G , such that the end points of edges in H are also contained in H . Strictly speaking, we have the following:

Definition 2.4: A graph H is a *subgraph* of G if $V(H) \subseteq V(G)$ and $E(H) \subseteq E(G)$ such that for all $e \in E(H)$ with $e = \langle u, v \rangle$, we have that $u, v \in V(H)$. When H is a subgraph of G , we write $H \subseteq G$.

As an example, Figure 2.5 shows a so-called **cubic graph** (i.e., 3-regular graph) with 8 vertices and three of its subgraphs.

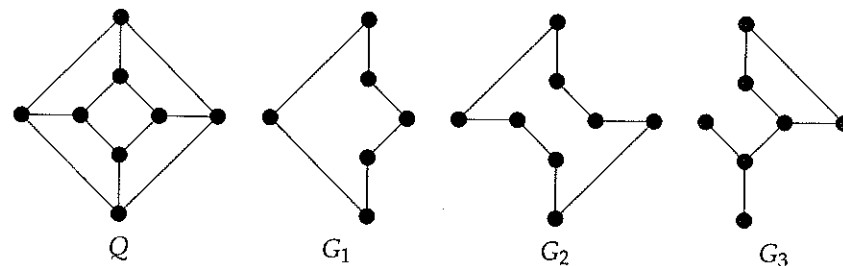


Figure 2.5: The cubic graph Q with 8 vertices and three subgraphs G_1 , G_2 , and G_3 .

When analyzing properties of graphs, it is often convenient to consider subgraphs formed by a specific subset of vertices. These are so-called *induced subgraphs*, which are constructed by taking a subset V^* of vertices and adding each edge from the original graph that connects two vertices from V^* . Formally, we have:

Definition 2.5: Consider a graph G and a subset $V^* \subseteq V(G)$. The *subgraph induced by V^** has vertex set V^* and edge set E^* defined by

$$E^* \stackrel{\text{def}}{=} \{e \in E(G) \mid e = \langle u, v \rangle \text{ with } u, v \in V^*\}$$

Likewise, if $E^* \subseteq E(G)$, the *subgraph induced by E^** has edge set E^* and a vertex set V^* defined by

$$V^* \stackrel{\text{def}}{=} \{u, v \in V(G) \mid \exists e \in E^* : e = \langle u, v \rangle\}$$

The subgraph induced by V^* or E^* is written as $G[V^*]$ or $G[E^*]$, respectively.

Clearly, every simple graph $G = (V, E)$ having n vertices can be seen as a subgraph of the complete graph K_n . Moreover, if we consider its complement $\bar{G} = (V, \bar{E})$, then the *union* of G and $\bar{G}$, that is, the graph with vertex set V and edge set $E \cup \bar{E}$, corresponds to K_n . This is what we have previously coined taking two graphs “together.”

Somewhat related to the notion of an induced subgraph is that of a line graph.

Definition 2.6: Consider a simple graph $G = (V, E)$. The *line graph* of G , denoted as $L(G)$ is constructed from G by representing each edge $e = \langle u, v \rangle$ from E by a vertex v_e in $L(G)$, and joining two vertices v_e and v_{e^*} if and only if edges e and e^* are incident with the same vertex in G .

To illustrate, consider the graph shown in Figure 2.6(a), containing four vertices and six edges. Its line graph, shown in Figure 2.6(b), consists of six vertices.

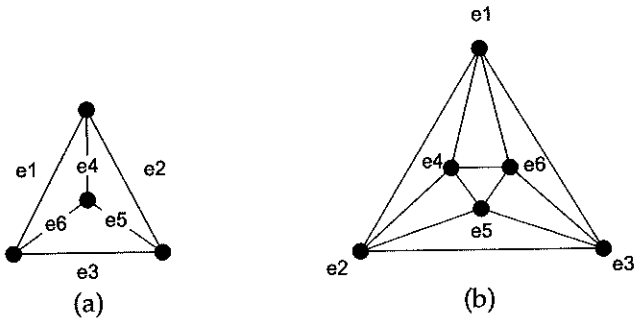


Figure 2.6: (a) A graph G and (b) its line graph $L(G)$.

Note 2.5 (Mathematical language)

Note that we used one of those awkward, yet precise mathematical statements when defining a subgraph induced by a set of edges. In this case, the mathematical statement

$$V^* \stackrel{\text{def}}{=} \{u, v \in V(G) \mid \exists e \in E^* : e = \langle u, v \rangle\}$$

should be translated into plain English as follows:

V^* is the set of vertices from $V(G)$ formed by the end points of edges in E^* .

If we would literally translate from math, we would have

V^* is defined by all vertices u and v from $V(G)$ for which there exists an edge in E^* that joins u and v .

When reading this second version, it is important to try to move away from all the math and come up with something like the first one, which is more intuitive and actually simpler.

A special induced subgraph is the one by which we simply remove a specific vertex, say v : $G[V(G) \setminus \{v\}]$. We came across this type of graph in our proof

of Theorem 2.2. Instead of using the notation $G[V(G) \setminus \{v\}]$ we will often simply write $G - v$. Likewise, if e is an edge, we will often write $G - e$ instead of $G[E(G) \setminus \{e\}]$. Similar simplified notations will be used when dealing with subsets of vertices or edges, respectively.

2.2 Graph representations

It should be clear from the presentation so far that graphs can be drawn in different ways, but also that when considering their formal definition, they are merely described in terms of vertices and edges. Let us now pay attention to how we can conveniently represent graphs. This issue is particularly important when we need to represent very large graphs for automated processing by computers.

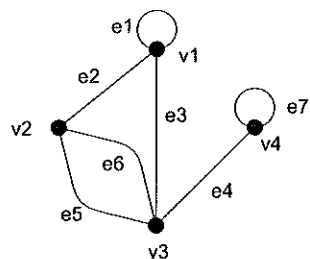
Data structures

There are different ways to represent graphs. Perhaps the most appealing one is to use an **adjacency matrix**. Consider a graph G with n vertices and m edges. Its adjacency matrix is nothing else but a table A with n rows and n columns with entry $A[i, j]$ denoting the number of edges joining vertex v_i and v_j . To illustrate, Figure 2.7 shows a simple graph with its accompanying adjacency matrix.

It is not difficult to see that the following properties hold:

- An adjacency matrix is *symmetric*, that is for all i, j , $A[i, j] = A[j, i]$. This property reflects the fact that an edge is represented as an *unordered* pair of vertices $e = \langle v_i, v_j \rangle = \langle v_j, v_i \rangle$.
- A graph G is simple if and only if for all i, j , $A[i, j] \leq 1$ and $A[i, i] = 0$. In other words, there can be at most one edge joining vertices v_i and v_j and, in particular, no edge joining a vertex to itself.
- The sum of values in row i is equal to the degree of vertex v_i , that is, $\delta(v_i) = \sum_{j=1}^n A[i, j]$.

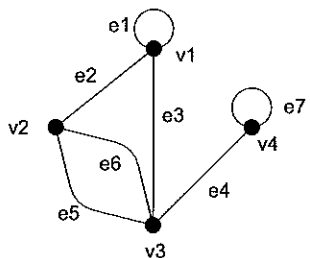
As an alternative, we can also use an **incidence matrix** of a graph as its representation. An incidence matrix M of graph G consists of n rows and m columns such that $M[i, j]$ counts the number of times that edge e_j is incident with vertex v_i . Note that $M[i, j]$ is either 0, 1, or 2: an edge can be only *not* incident with vertex v_i , it has vertex v_i as exactly one of its end points, or is a loop joining vertex v_i with itself. Figure 2.8 shows the incidence matrix for the graph from Figure 2.7. Again, the following properties are easy to verify:



	v_1	v_2	v_3	v_4
v_1	2	1	1	0
v_2	1	0	2	0
v_3	1	2	0	1
v_4	0	0	1	2

Figure 2.7: A graph with its associated adjacency matrix.

- A graph G has no loops if and only if for all i, j , $M[i, j] \leq 1$.
- The sum of all values in row i is equal to the degree of vertex v_i . In mathematical terms, this is expressed as $\forall i : \delta(v_i) = \sum_{j=1}^m M[i, j]$.
- Because each edge has exactly two, not necessarily distinct end points, we know that for all j , $\sum_{i=1}^n M[i, j] = 2$.



	e_1	e_2	e_3	e_4	e_5	e_6	e_7
v_1	2	1	1	0	0	0	0
v_2	0	1	0	0	1	1	0
v_3	0	0	1	1	1	1	0
v_4	0	0	0	1	0	0	2

Figure 2.8: A graph with its associated incidence matrix.

One of the problems with using either an adjacency matrix or an incidence matrix is that without further optimizations, the total number of elements for representing a graph is $n \times n$ or $n \times m$, respectively. This is not very efficient when having to deal with very large graphs, especially when the number of edges is relatively small. To see why this is true, consider the representation of an adjacency matrix in a computer. Assume that we use only a single byte to count the number of edges joining a pair of vertices. Without any further optimizations, a graph with 100,000 vertices would require a total of $100,000 \times 100,000$ bytes of storage, that is, close to 10 Gbyte. Using an incidence matrix and assuming a total of 250,000 edges, a straightforward, nonoptimized representation would require close to 25 Gbytes of

storage. Both representations, even when applying all kinds of storage optimizations, generally tend to be rather inefficient.

An often more efficient representation, and used in practice, is that of an **edge list**. In this case, we merely list the edges of a graph G by specifying for each edge which vertices it is incident with. Note that this representation grows linearly with the number of edges. For example, the edge-list representation of the graph from Figure 2.8 is:

$$(\langle v_1, v_1 \rangle, \langle v_1, v_2 \rangle, \langle v_1, v_3 \rangle, \langle v_2, v_3 \rangle, \langle v_2, v_3 \rangle, \langle v_3, v_4 \rangle, \langle v_4, v_4 \rangle)$$

In particular, with m edges, we would need to store only $2 \cdot m$ data items. Assuming that a vertex can be represented by four bytes, this means that for our example graph with 100,000 vertices and 250,000 edges, we would need only close to 2 Mbytes of storage. In practice, this number will be larger because we need additional data structures to easily navigate through the edge list. Nevertheless, the total amount of required storage will generally stay significantly less than what is needed for an adjacency or incidence matrix.

It should be clear that by simply going through this list, we also find the vertices of the associated graph, provided that each vertex is incident with at least one edge. In practice, an edge list is often accompanied by a list of vertices, for example, to describe attached labels (such as "v1").

Graph isomorphism

An important observation is that all these representations are independent of the way that we draw a graph. Consider the graphs shown in Figure 2.9. No matter whether we represent each graph by its adjacency matrix, incidence matrix, or edge list, if we properly attach labels to vertices and edges, we will find that their respective representations are exactly the same. As a consequence, they should also be considered to be the same. This notion of similarity is formalized through what is known as graph isomorphism.

Definition 2.7: Consider two graphs $G = (V, E)$ and $G^* = (V^*, E^*)$. G and G^* are **isomorphic** if there exists a one-to-one mapping $\phi : V \rightarrow V^*$ such that for every edge $e \in E$ with $e = \langle u, v \rangle$, there is a unique edge $e^* \in E^*$ with $e^* = \langle \phi(u), \phi(v) \rangle$.

Stated differently, two graphs G and G^* are isomorphic if we can uniquely map the vertices and edges of G to those of G^* such that if two vertices were joined in G by a number of edges, their counterparts in G^* will be joined by the same number of edges.

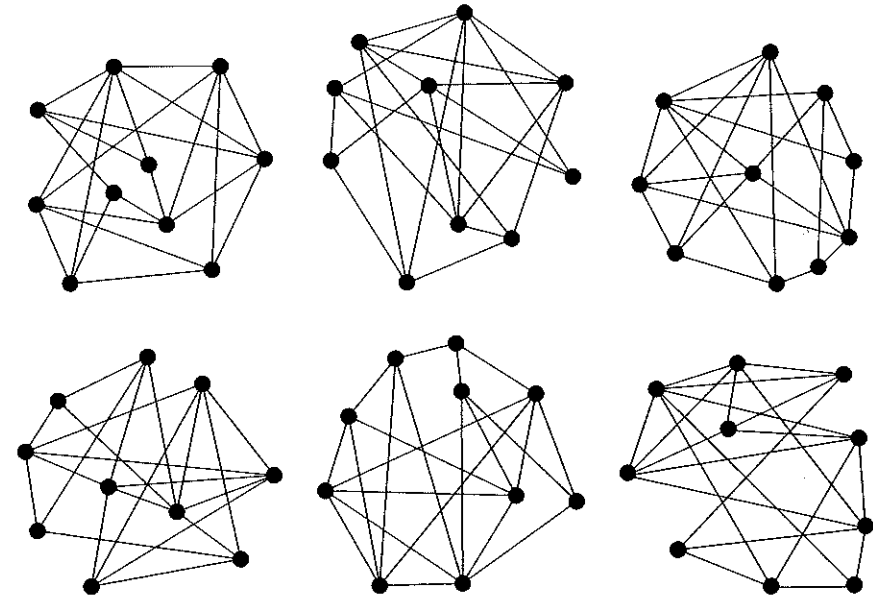


Figure 2.9: Six different drawings of graphs with the same representation, that is, isomorphic graphs.

Note 2.6 (Mathematical language)

Couldn't we just talk about the *same* graphs, you might wonder, instead of using a term like *isomorphism*? However, "isomorphism" is a well-defined mathematical concept that is used for more than just graphs. In essence, it is used in those situations where we are dealing with sets (like vertices), and that the elements in those sets are somehow organized in a specific way. Isomorphism is then used to express that two sets have essentially the same elements when you ignore labeling issues, but also that their organization is the same. An isomorphism is then a *structure-preserving* mapping between two sets.

In many cases, checking whether two graphs are isomorphic is relatively simple as there are a number of important *necessary* requirements that need to be fulfilled. For example, it should be obvious that the two graphs need to have the same number of vertices and edges in order to be isomorphic. A stronger requirement is that they have the same ordered degree sequence. This may seem obvious, but if we want to be precise, showing the obvious may turn out to be more cumbersome than expected. Let's consider the following formal formulation.

Theorem 2.3: If two graphs G and G^* are isomorphic, then their respective ordered degree sequences should be the same.

Proof. Let ϕ be the one-to-one mapping by which G and G^* are known to be isomorphic. Consider vertex u from G and its adjacent vertices $v_1, \dots, v_k$. By definition, each edge $e_i = \langle u, v_i \rangle$ incident with u in G is mapped to a unique edge $e_i^* = \langle \phi(u), \phi(v_i) \rangle$ in G^* . Because each edge e_i^* is incident with $\phi(u)$, we must have that $\delta(u) = \delta(\phi(u))$.

Now consider a vertex $v^* \in V(G^*)$ that is adjacent to $\phi(u)$. By definition of isomorphism, we know that the edge $e^* = \langle \phi(u), v^* \rangle$ must uniquely map to an edge $e = \langle \phi^{-1}(\phi(u)), \phi^{-1}(v^*) \rangle$ in G , where ϕ^{-1} denotes the inverse mapping of ϕ . Because ϕ is a one-to-one mapping, we also know that $\phi^{-1}(\phi(u)) = u$, and thus that $e = \langle u, \phi^{-1}(v^*) \rangle$. In other words, every edge incident with $\phi(u)$ in G^* will be incident with u in G . This means that $\delta(\phi(u)) \leq \delta(u)$.

We conclude that $\delta(u) = \delta(\phi(u))$ for all vertices of G , implying that the ordered degree sequences of G and G^* should be the same. $\square$

Unfortunately, this theorem gives us only a *necessary* condition for two graphs to be isomorphic, yet it is not a *sufficient* condition. In other words, if two graphs have the same ordered degree sequence, then that fact alone is not sufficient to conclude that they are also isomorphic. Yet to be isomorphic, it is *necessary* for their respective ordered degree sequences to be the same.

Note 2.7 (Mathematical language)

The difference between *necessary* and *sufficient* conditions seems an obvious one, yet they are surprisingly often confused in mathematical proofs. Formally, in graph theory, conditions are used to prove properties of graphs. When a condition C is said to be *necessary*, this means that a property P can hold *only if* C is met. When a condition C is said to be *sufficient*, this means that *if* C is met, then property P will hold true. And indeed, when property P is true *if and only if* condition C is met, indicates that C is a *necessary and sufficient* condition for property P to be valid.

To illustrate, consider the graphs from Figure 2.4(a), which are shown again in Figure 2.10. Although they have the same ordered degree sequence, they are not isomorphic. One way of seeing this is that the two vertices with degree 3 are adjacent to one another in G , but not in G^* . (There are other structural differences, yet explaining these requires the introduction of more graph concepts, which we defer until later.)

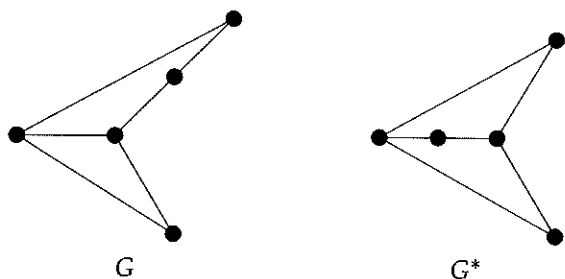


Figure 2.10: Two nonisomorphic graphs with the same ordered degree sequence.

The bad news is that there are no known easy sufficient conditions that will tell us in general whether two graphs are isomorphic or not. Essentially, this means that once we have found that all necessary conditions have been fulfilled, we will have to resort to a trial-and-error method. For example, with the graphs from Figure 2.10, we were able to successfully consider whether the highest-degree vertices were adjacent in both graphs. In other cases, however, we may have to look at other properties.

Note 2.8 (More information)

In the worst case, we may have to resort to an *exhaustive* method. Consider a graph G with n vertices $\{v_1, v_2, \dots, v_n\}$, and a graph G^* also with n vertices. To check for isomorphism, we need to find a one-to-one mapping between these two vertex sets. With an exhaustive approach, we simply go through all possible mappings to see if there is one that establishes isomorphism. Unfortunately, there may be quite a few mappings that we need to check. To be precise, there are potentially $n!$ mappings to consider, where

$$n! \stackrel{\text{def}}{=} n \cdot (n-1) \cdot (n-2) \cdots 2 \cdot 1$$

(to be pronounced as *n factorial*). This is relatively easy to see as follows. For any mapping, we have n choices for mapping v_1 to one of the vertices from G^* . After that, there are $n-1$ possibilities left for mapping v_2 to a vertex from G^* , and then another $n-2$ for mapping v_3 , and so on. Finally, after having made a choice for each vertex $v_1, v_2, \dots, v_{n-1}$, we have only one more option left for v_n .

Checking $n!$ mappings is no pleasure game—consider the following table:

n	$n!$	n	$n!$	n	$n!$
1	1	6	720	11	39,916,800
2	2	7	5040	12	479,001,600
3	6	8	40,320	13	6,227,020,800
4	24	9	362,880	14	87,178,291,200
5	120	10	3,628,800	15	1,307,674,368,000

In fact, for large n , its factorial can be approximated by

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

which reaches amazingly high numbers even for relatively small values of n . There is also no chance that brute-force computations with a computer are going to bring any serious help here. For example, if a computer were able to check whether one specific mapping could establish isomorphism between two graphs in only 1 nanosecond (which is 10^{-9} seconds), it would still take about 500 years to go through all possible mappings for two 25-vertex graphs. More cleverness is needed.

We note that algorithms do exist that can efficiently test isomorphism for many graphs up to approximately 100 vertices, with perhaps the fastest one being nauty devised by McKay [1980]. Also, efficient algorithms exist for graphs for which the maximal vertex degree is known to be bound by a constant [Luks, 1982].

2.3 Connectivity

In all the graphs we have considered so far, each vertex v could be reached from any other vertex w in the sense that we could indicate a chain of adjacent vertices from v to w . In this section, we will take a closer look at this important concept of connectivity. We start with some basic terminology:

Definition 2.8: Consider a graph G . A (v_0, v_k) -walk in G is an alternating sequence $[v_0, e_1, v_1, e_2, \dots, v_{k-1}, e_k, v_k]$ of vertices and edges from G with $e_i = \langle v_{i-1}, v_i \rangle$. In a *closed walk*, $v_0 = v_k$. A *trail* is a walk in which all edges are distinct; a *path* is a trail in which also all vertices are distinct. A *cycle* is a closed trail in which all vertices except v_0 and v_k are distinct.

Using the notion of a path, we define a graph to be connected when there is a path between each pair of distinct vertices. Formally, we have:

Definition 2.9: Two distinct vertices u and v in graph G are *connected* if there exists a (u, v) -path in G . G is *connected* if all pairs of distinct vertices are connected.

Clearly, all the graphs we have considered so far are indeed connected. However, there is no reason to assume that a graph is always connected. If we take a look at the definition of a graph, there is nothing there that states that all vertices should be connected. Intuitively, this means that a graph could also consist as a collection of components, where each component is a connected subgraph. This can be made precise as follows:

Definition 2.10: A subgraph H of G is called a *component* of G if H is connected and not contained in a connected subgraph of G with more vertices or edges. The number of components of G is denoted as $\omega(G)$.

Note that a component is not just a subgraph: it is a *maximal, connected* subgraph. In other words, if we would consider a subgraph H of a graph G and would find that there is a vertex not in H that is connected to a vertex in H , then H is, by definition, not a component. Maximality also incorporates edges, meaning that if an edge e joins two vertices in G , e should be contained in H .

The notion of connectivity is important, notably when considering the *robustness* of networks. Robustness in this context means how well the network stays connected when we remove vertices or edges. For example, as we mentioned in Chapter 1, the Internet can be viewed as a (huge) graph in which routers form the vertices and communication links between routers the edges. In a formal sense, the Internet is connected. However, if it were possible to partition the network into multiple components by removing only a single vertex (i.e., router) or edge (i.e., communication link), we could hardly claim the Internet to be robust. In fact, it is extremely important for networks such as the Internet to be able to sustain serious attacks and failures by which routers and links are brought down, such that connectivity is still guaranteed.

There are many networks for which robustness in one way or another plays an important role. Let us now formalize this notion by considering what are known as vertex and edge cuts.

Definition 2.11: For a graph G let $V^* \subset V(G)$ and $E^* \subset E(G)$. V^* is called a *vertex cut* if $\omega(G - V^*) > \omega(G)$. If V^* consists of a single vertex v , then v is called a *cut vertex*. Likewise, if $\omega(G - E^*) > \omega(G)$ then E^* is called an *edge cut*. If E^* consists of only a single edge e , then e is known as a *cut edge*.

Note that we have used the notation $G - V^*$ to indicate the induced subgraph $G[V(G) \setminus V^*]$. What the definition states is that V^* is a vertex cut of a connected graph if the removal of vertices in V^* from G will make G disintegrate into several components. In other words, G will become disconnected. Analogously, an edge cut of G is a collection of edges that will make G fall apart into multiple components when those edges are removed. In the definition given above, we have used the simpler notation $G - E^*$ to indicate the induced subgraph $G[E(G) \setminus E^*]$.

Of particular interest is the *minimal* vertex cut for a connected graph. In other words, how many vertices do we need to remove from a connected graph before it becomes disconnected? An important observation is the following. Let $\kappa(G)$ denote the size of a minimal vertex cut for graph G , and

likewise, $\lambda(G)$ the size of a minimal edge cut. As it turns out, $\kappa(G) \leq \lambda(G)$, but also that $\lambda(G)$ is less or equal to the minimal vertex degree. Using the notation $\min S$ to denote the smallest value found among the elements in set S , these properties are formulated in the following important theorem.

Theorem 2.4: $\kappa(G) \leq \lambda(G) \leq \min\{\delta(v) | v \in V(G)\}$

Proof. That $\lambda(G) \leq \min\{\delta(v) | v \in V(G)\}$ is easy to see. Consider a vertex u with minimal degree, that is, $\delta(u) = \min\{\delta(v) | v \in V(G)\}$. If we simply remove the $\delta(u)$ edges incident with u , then u will become isolated, and certainly the resulting graph will have at least one more component than it had before (namely the one consisting only of u).

To prove that $\kappa(G) \leq \lambda(G)$, consider a graph G with $\lambda(G) = k$ and let $E^* = \{e_1, e_2, \dots, e_k\}$ be a minimal edge cut of G , with $e_i = \langle u_i, v_i \rangle$. Let U denote the set of vertices $\{u_1, \dots, u_k\}$ and V the set $\{v_1, \dots, v_k\}$. Note that in this case, the vertices in either set need not be distinct. The graph $G - E^*$ will fall apart into exactly two components, say G_1 and G_2 (we leave it to you to show that this is indeed true). If G_1 contains a vertex u distinct from any u_i , as shown in Figure 2.11(a), then clearly removing all vertices in U will disconnect u from any vertex in G_2 , so that $\kappa(G) \leq k$.

If there is no such vertex u , then assume that $V(G_1) = U$. Consider vertex u_1 . We know that u_1 is adjacent to d_1 vertices from G_1 , and each of these neighbors in G_1 is adjacent to a vertex from V . Let E_1^* be a set of edges from E^* joining vertices from the d_1 neighbors of u_1 and exactly one vertex from V . Likewise, let E_2^* be the d_2 edges from E^* incident with u_1 . This situation is shown in Figure 2.11(b). Obviously, $d_1 + d_2 = |E_1^* \cup E_2^*| \leq |E^*|$. Also, the $d_1 + d_2$ neighboring vertices of u_1 form a vertex cut, shown as open circles in Figure 2.11(b). This also means that $\kappa(G) \leq d_1 + d_2 \leq |E^*| = \lambda(G)$, completing the proof. $\square$

A graph G for which $\kappa(G) \geq k$ for some k is said to be **k-connected**. Likewise, graph G is **k-edge-connected** if $\lambda(G) \geq k$. Finally, a graph for which $\kappa(G) = \lambda(G) = \min\{\delta(v) | v \in V(G)\}$ is said to be **optimally connected**.

Note 2.9 (Study tip)

The previous proof, and notably proving that $\kappa(G) \leq \lambda(G)$, is a typical example where graph theory requires insight. The proof is not obvious, and it can certainly not be expected that an undergraduate student would be able to devise it from scratch. What is important, however, is that the proof itself is understood well. To this end, you are encouraged to start with *reproducing* proofs, as this will enforce you to carefully think about every step that is taken. Simply being

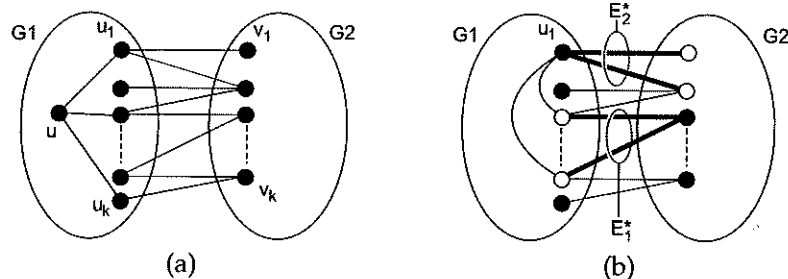


Figure 2.11: The two scenarios for the proof of Theorem 2.4.

able to reproduce proofs is a well-known technique to successfully study graph theory.

What Theorem 2.4 tells us is that every graph is at most $\delta_{\min}$ -edge connected, and at most $\delta_{\min}$ -connected, where $\delta_{\min} = \min\{\delta(v) | v \in V(G)\}$. We showed this for edge connectivity. Vertex connectivity is also easy to see: simply remove the $\delta_{\min}$ vertices adjacent to a vertex of degree $\delta_{\min}$ and the latter becomes disconnected. Of course, finding a lower bound for k is more interesting, but this turns out to be a relatively difficult problem to solve. Without going into the rather intricate details, we can say something about a lower bound for k by considering the notion of path independence.

Definition 2.12: Consider a graph G and a collection $\mathbf{P}$ of (u, v) -paths in G , with $u, v \in V(G)$. $\mathbf{P}$ is *vertex independent* if for all (u, v) -paths $P_1, P_2 \in \mathbf{P}$ we have that $V(P_1) \cap V(P_2) = \{u, v\}$. The collection is *edge independent* if for all its (u, v) -paths P_1 and P_2 , we have that $E(P_1) \cap E(P_2) = \emptyset$.

In other words, two (u, v) -paths P_1 and P_2 are vertex independent if they share only the vertices u and v , and are edge independent if they have no edge in common. Using path independence, we now come to one of the more fundamental theorems in graph theory, formulated by the Austrian mathematician Karl Menger.

Theorem 2.5 (Menger): Let G be a connected graph and u and v two nonadjacent vertices in G . The minimum number of vertices in a vertex cut that disconnects u and v is equal to the maximum number of pairwise vertex-independent paths between u to v . Analogously, the minimum number of edges in an edge cut that disconnects u and v , is equal to the maximum number of pairwise edge-independent paths between u and v .

We omit the proof, and instead refer the interested reader to Bondy and Murty [2008], Diestel [2005], or West [2001].

Note 2.10 (Mathematical language)

Menger's theorem should be read carefully: it mentions *pairwise independent paths*. In this case, the adjective *pairwise* is used to make clear that we should always consider pairs of paths when considering independence. And indeed, this makes sense when you would consider trying to count the number of *independent paths*: being an independent path can only be relative to another path.

To complete the story, also note that the theorem is all about counting the number of (u, v) -paths, and not the number of *pairs* of such paths. In other words, *pairwise* is an adjective to *independent*, and not to *paths*.

It is not difficult to see that Menger's theorem leads to the following corollary:

Corollary 2.2: A graph G is k -connected if and only if any two distinct vertices are connected by at least k pairwise vertex-independent paths. G is k -edge connected if and only if any two distinct vertices are connected by at least k pairwise edge-independent paths.

Of particular interest is the following one:

Corollary 2.3: Each edge of a 2-edge-connected graph lies on a cycle.

This corollary actually follows from the previous one, which states (for the special case $k = 2$) that a graph is 2-edge-connected if and only if any two distinct vertices are connected by at least 2 pairwise edge-independent paths. The latter, of course, together form a cycle. We will use this corollary in the next chapter when discussing so-called directed graphs.

Intuitively, it should be clear that for any simple graph G a higher value of $\kappa(G)$, i.e., the size of a minimal vertex cut, implies that more edges are needed. We have just seen that in every k -connected graph each vertex will have at least k incident edges. Knowing that $\sum \delta(v) = 2 \cdot m$, this means that for a graph with n vertices, we would need $\frac{1}{2} \sum \delta(v)$ and thus at least $\frac{1}{2} \sum k = \frac{1}{2} n \cdot k$ edges. But what is the minimal number of edges for a graph to be k -connected? This question brings us to a so-called Harary graph:

Definition 2.13: A Harary graph $H_{k,n}$ is a k -connected simple graph with n vertices and with a minimal number of edges.

What we now need to figure out is actually how many edges an Harary graph has. We will show that $H_{k,n}$ has exactly $\lceil k \cdot n/2 \rceil$ edges, that is, the smallest natural number of edges greater or equal to $k \cdot n/2$. To this end, we

label the vertices in $H_{k,n}$ as $\{0, 1, \dots, n-1\}$ and organize them graphically as a circle. Following Bondy and Murty [1976], we consider the following three cases for combinations of k and n .

k is even: We construct $H_{k,n}$ by joining each vertex i to its $k/2$ closest left-hand (i.e., clockwise) neighbors and its $k/2$ closest right-hand (i.e., counterclockwise) neighbors¹.

k is odd, n is even: In this case, we construct $H_{k-1,n}$ and add $n/2$ edges by joining vertex i to its left-hand neighbor at distance $\frac{n}{2}$ (with $0 \leq i < \frac{n}{2}$). In other words, we add edges $\langle 0, \frac{n}{2} \rangle, \langle 1, 1 + \frac{n}{2} \rangle, \langle 2, 2 + \frac{n}{2} \rangle, \dots, \langle \frac{n-2}{2}, n-1 \rangle$.

k is odd, n is odd: In this case, we again first construct $H_{k-1,n}$ and then add the $(n+1)/2$ edges $\langle 0, \frac{n-1}{2} \rangle, \langle 1, 1 + \frac{n-1}{2} \rangle, \dots, \langle \frac{n-1}{2}, n-1 \rangle$.

To clarify the construction of these graphs, Figure 2.12 shows graphs $H_{4,8}$, $H_{5,8}$, and the construction of $H_{5,9}$ from $H_{4,9}$.

Note 2.11 (More information)

At first sight, constructing Harary graphs seems to be one of those typical mathematical topics: nice, but it looks as if someone got carried away a bit. In fact, Harary graphs address a very relevant question in communication networks: trading off the costs between reliability and the number of communication links. A communication network constructed as an Harary graph $H_{k,n}$ tells us that we can remove up to k vertices before the network becomes partitioned. This means that if we are considering networks that are designed to disseminate data to every node, Harary graphs will give us the means to make them just as robust as we want them to be, yet with a minimal number of links. There are a number of variations on this theme, as explained by McQuillan [1977].

Admittedly, when first thought of, people considered the monetary costs of a communication link. With the robustness of the Internet, the problem seems to be less relevant. However, suppose we formulate costs in terms of how quickly data is disseminated. As we shall discuss in Chapter 8, we often want to construct an artificial, or virtual network on top of an existing communication network such as the Internet. In that case, we can shape the network as we like. As discussed by Jenkins and Demers [2001], Harary graphs are useful for constructing virtual networks that will optimally disseminate data in a group of nodes, while keeping that group k -connected.

Now that we have the procedure to construct Harary graphs, we need to show that they indeed have a minimal number of edges while maintaining the property of being k -connected. We first prove connectivity.

¹Of course, being a left-hand or right-hand neighbor makes sense only if we assume that a vertex has an orientation. In our example, we orient a vertex toward the middle of the ring.

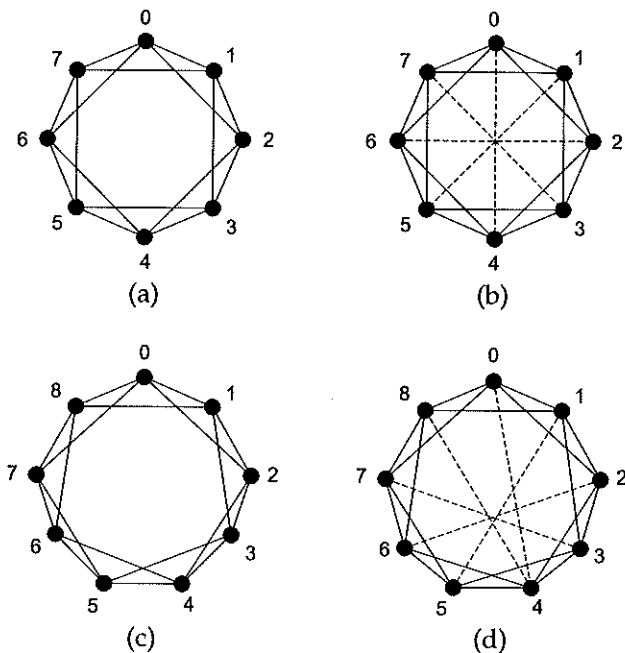


Figure 2.12: Various Harary graphs: (a) $H_{4,8}$, (b) $H_{5,8}$, (c) $H_{4,9}$, and (d) $H_{5,9}$. Dashed edges are the ones added to obtain $H_{5,8}$ from $H_{4,8}$, and $H_{5,9}$ from $H_{4,9}$, respectively.

Theorem 2.6: The Harary graph $H_{k,n}$ is k -connected.

Proof. Let us first consider the case that k is even. Our proof is completed if we can show that there is no vertex cut with fewer than k vertices. To this end, let us assume that such a set W does exist. If we can then prove that this assumption can never hold, we will have completed our proof (we come back to this method of proving a theorem below).

To this end, let vertices i and j belong to different components of $H_{k,n} - W$ (i.e., $G[V(H_{k,n}) \setminus W]$). Consider the set $N_{i \rightarrow j}$ of left-hand neighbors of i , including i : $\{i, i+1, \dots, j-1, j\}$, and likewise its right-hand neighbors $N_{i \leftarrow j} = \{j, j+1, \dots, i-1, i\}$. In both cases, addition is taken modulo n . Let $W_{i \rightarrow j} \stackrel{\text{def}}{=} W \cap N_{i \rightarrow j}$ and $W_{i \leftarrow j} \stackrel{\text{def}}{=} W \cap N_{i \leftarrow j}$ (meaning that $W = W_{i \rightarrow j} \cup W_{i \leftarrow j}$). We know that $|W| < k$, so we must have that either $|W_{i \rightarrow j}| < k/2$ or $|W_{i \leftarrow j}| < k/2$, as is illustrated in Figure 2.13. Assume that $|W_{i \rightarrow j}| < k/2$.

Now consider an arbitrary vertex u in $H_{k,n} - W$, lying on, say, segment S_1 (see Figure 2.13). We know that u is adjacent to $k/2$ consecutive vertices in either direction. As a consequence, removing less than $k/2$ vertices as is

done through $W_{i \rightarrow j}$ will still allow us to reach any vertex v on segment S_2 . In other words, $H_{k,n} - W$ will remain connected, contradicting our assumption that W was a vertex cut. $\square$

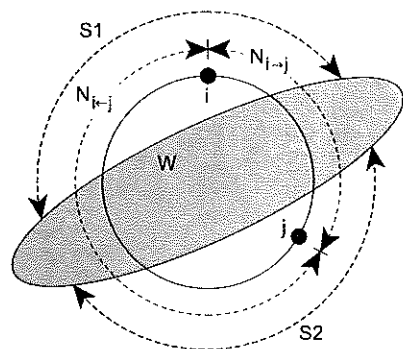


Figure 2.13: Illustration that $|W \cap N_{i \rightarrow j}| < k/2$ or $|W \cap N_{j \rightarrow i}| < k/2$.

Note 2.12 (Proof techniques)

We have just encountered our first **proof by contradiction**. This method is widely applied and you should definitely familiarize yourself with it. The principle is fairly straightforward: if you want to prove some statement P to be true, then in a proof by contradiction you *assume* that P cannot hold and subsequently show that this assumption will lead to something nonsensical. Nonsense can then only mean that your assumption was incorrect.

There is another important principle that surfaced in the previous proof, which is known as the **pigeonhole principle**. This principle simply states that if n items need to be spread over $m < n$ boxes, then there will be at least one box containing more than one item. How did we apply this principle? In our proof, we noted that the set W contained less than k elements and that we split it into two parts $W_{i \rightarrow j}$ and $W_{j \rightarrow i}$. The pigeonhole principle tells us that at least one of these two sets must have less than $k/2$ elements.

What remains is to show that a Harary graph also has a minimal number of edges. For any k -connected graph, we now know that each vertex v has a degree $\delta(v) \geq k$. Let $m_k(n)$ denote the minimal number of edges for any simple, k -connected graph G . Because $|E(G)| = \frac{1}{2} \sum_{v \in V(G)} \delta(v) \geq \frac{1}{2} \sum_{v \in V(G)} \delta_{\min} \geq \frac{n \cdot k}{2}$, we know that $m_k(n) \geq \frac{n \cdot k}{2}$. It is not difficult to verify that $|E(H_{k,n})| = \frac{n \cdot k}{2}$, meaning that an Harary graph indeed has a minimal number of edges.

2.4 Drawing graphs

As the saying goes, a picture does often say more than a 1000 words. This certainly also holds for drawing graphs. We have already seen various examples of how the same graph can be drawn in different ways. As it turns out, this subject is so important that researchers have spent considerable effort on devising algorithms for drawing graphs. In this section we take a closer look at some of the results.

Graph embeddings

To illustrate why properly drawing graphs may be important, consider the graphs from Figure 2.14, which shows the **Petersen graph**, a particular 3-regular graph. Clearly, by just looking at these drawings, it is not obvious that we are dealing with the same graph (i.e., same in the sense of isomorphic). This instantly brings up the issue of what makes a good drawing of a graph. In this example, either Figure 2.14(a) or (b) is arguably the best one.

Formally, when drawing graphs we are considering so-called **graph embeddings**: a representation of a graph on a surface where vertices are associated with points on that surface. In practice, we always consider the two-dimensional plane, but note that embeddings in three dimensions are also possible.

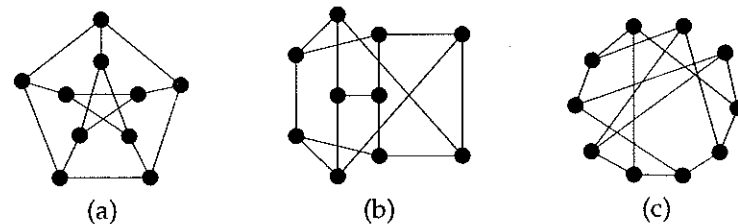


Figure 2.14: Three different drawings of the Petersen graph.

A commonly applied embedding is the **circular embedding**. In this case, the vertices are placed at evenly spaced points on a circle, as illustrated by the Petersen graph in Figure 2.14(c). The advantage of this representation is that no three vertices ever lie on the same straight line, or, in other words, are ever collinear. This has the effect that each edge can be easily drawn such that it remains visible in the drawing. This is an important property, notably when dealing with so-called **random graphs** in which pairs of vertices are connected by randomly chosen edges. In that case, it is generally important

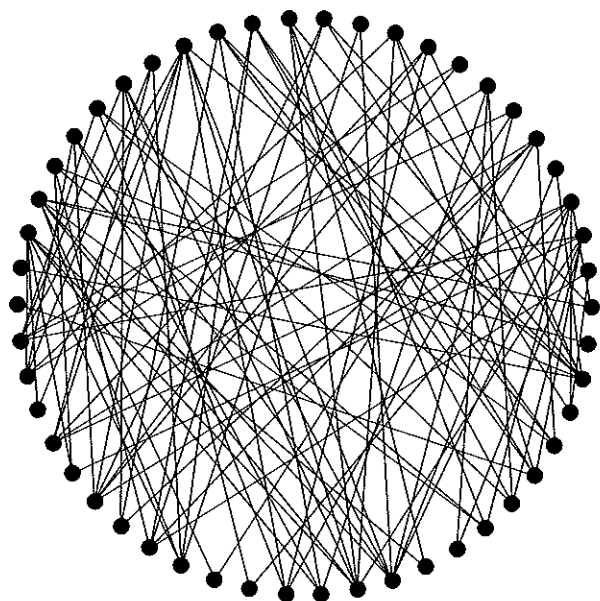


Figure 2.15: A random graph with 50 vertices and 103 edges. The circular embedding allows to draw each edge as a straight line that remains (reasonably) visible.

to see all edges. Figure 2.15 shows such a random graph with 50 vertices and 103 edges.

There are other useful embeddings to consider. For example, an important class is formed by **bipartite graphs**: graphs of which the set of vertices can be partitioned into two subsets such that no edge is incident to vertices from the same subset. In other words, each edge is incident to a vertex from either set:

Definition 2.14: A graph G is **bipartite** if $V(G)$ can be partitioned into two disjoint subsets V_1 and V_2 such each edge $e \in E(G)$ has one end point in V_1 and the other in V_2 : $E(G) \subseteq \{e = \langle u_1, u_2 \rangle | u_1 \in V_1, u_2 \in V_2\}$.

Bipartite graphs are sometimes conveniently drawn as **ranked embeddings**. To explain, reconsider the graph from Figure 2.15. Although it is not obvious from the drawing at all, it turns out that this graph is actually bipartite. We can discover this by considering what the distance is between a given vertex v and each other vertex. The distance between two vertices v and w is informally expressed as the minimal number of edges between v and w (also called the **shortest path** to which we return in Chapter 4). This leads to a group of vertices at distance 1 (i.e., vertices adjacent to v ,

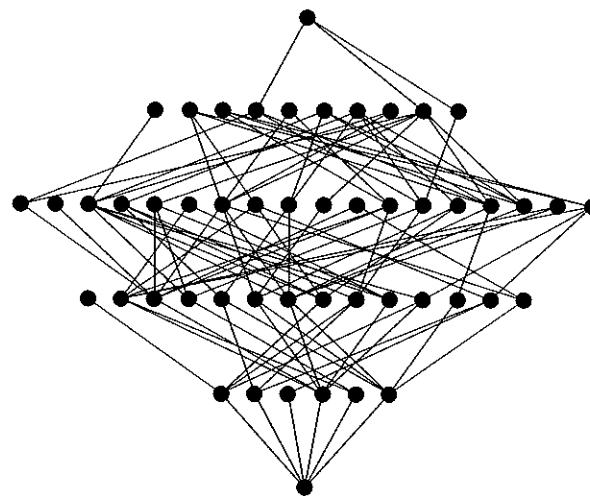


Figure 2.16: A ranked embedding of the graph from Figure 2.15.

which we had defined as its neighbors), at distance 2 (vertices adjacent to the vertices adjacent to v), etc. Now consider Figure 2.16 for which we have selected an arbitrary vertex v from G and subsequently (1) ranked all other vertices according to their distance, and (2) placed all vertices at the same distance along the same vertical line. What we observe is that there are no edges between vertices at the same distance. This can only mean that G is bipartite. In fact, in this example the set of vertices can be partitioned into six disjoint subsets.

These examples illustrate that examining graphs through visual inspection requires the use of computer tools. What these tools invariably do is compute vertex positions in the two-dimensional plane according to some simple or complex criterion. Circular and ranked embeddings are relatively simple. More complex ones involve spreading vertices far apart while still keeping connected ones close to each other. An example of such an approach is a **spring embedding** [Eades, 1984]. In this case, the vertices are modeled as rings connected by springs. Initially, the vertices are positioned randomly in the two-dimensional plane, after which the springs do their work by trying to reach an equilibrium. To illustrate, Figure 2.17 shows a number of steps by which the randomly positioned vertices in Figure 2.17(a) are gradually brought into an equilibrium.

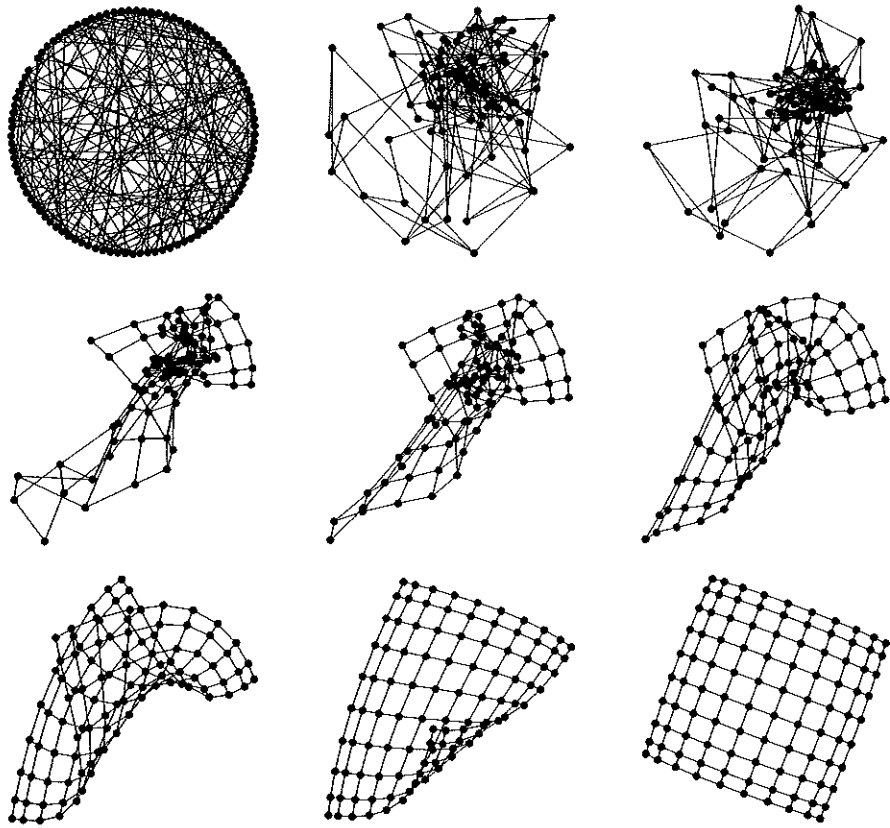


Figure 2.17: The evolution of applying a spring embedding to a graph.

Note 2.13 (More information)

As illustrated by Figure 2.17, spring embeddings can lead to an appealing visualization of a graph. Let's take a closer look at how the approach works. As stated previously, Eades [1984] proposed to represent each vertex as a ring and each edge as a spring. Each vertex u is initially positioned in a two-dimensional plane, with coordinates (u_x, u_y) . Each spring $e = \langle u, v \rangle$ exerts an attracting force $F_{att}(u, v)$ between the vertices u and v it joins according to the formula

$$F_{att}(u, v) \stackrel{\text{def}}{=} \begin{cases} 2 \log(d(u, v)) & \text{if } u \text{ and } v \text{ are adjacent} \\ 0 & \text{otherwise} \end{cases}$$

where $d(u, v)$ is the length of the spring between u and v . This length corre-

sponds to the distance between u and v defined as:

$$d(u, v) \stackrel{\text{def}}{=} \sqrt{(u_x - v_x)^2 + (u_y - v_y)^2}$$

Note that there is nothing special to this definition of distance: it is a direct application of the Pythagorean theorem. Besides an attracting force, Eades also introduces a repelling force $F_{rep}(u, v)$ between nonadjacent vertices u and v , defined as:

$$F_{rep}(u, v) \stackrel{\text{def}}{=} \begin{cases} 0 & \text{if } u \text{ and } v \text{ are adjacent} \\ 1/\sqrt{d(u, v)} & \text{otherwise} \end{cases}$$

With these forces defined, we now have a system of attracting and repelling vertices. When the vertices are placed randomly, it should be clear that there will generally be a lot of pushing and pulling going on. In particular, if the resulting pushing and pulling forces on a vertex are not equal, we can expect the vertex to be moving to a position in which there is more equilibrium. This behavior can be simulated by means of the following algorithm:

Algorithm 2.1 (Spring embedding):

1. Place the vertices at random locations
2. For each vertex u , calculate the resulting forces in the x and y direction, respectively:

$$\begin{aligned} F_x(u) &\stackrel{\text{def}}{=} \sum_{v \neq u} (F_{att,x}(u, v) - F_{rep,x}(u, v)) \\ F_y(u) &\stackrel{\text{def}}{=} \sum_{v \neq u} (F_{att,y}(u, v) - F_{rep,y}(u, v)) \end{aligned}$$

3. Reposition vertex u according to:

$$u_x \leftarrow u_x + 0.1 \cdot F_x(u) \quad \text{and} \quad u_y \leftarrow u_y + 0.1 \cdot F_y(u)$$

4. Goto to Step 2. Stop after M iterations.

$F_{att,x}(u, v)$ is the attracting force in the x direction from neighboring vertex v , computed as:

$$F_{att,x}(u, v) \stackrel{\text{def}}{=} F_{att}(u, v) \cdot \frac{|v_x - u_x|}{d(u, v)}$$

The respective definition of $F_{att,y}(u, v)$, $F_{rep,x}(u, v)$, and $F_{rep,y}(u, v)$ is analogous.

Furthermore, we have used the notation " $x \leftarrow S$ " to denote that x takes on the value resulting from evaluating expression S . So, in our example, u_x is adjusted by $0.1 \cdot F_x(u)$ units. In practice, a state very close to equilibrium is reached for at most $M = 100$ iterations.

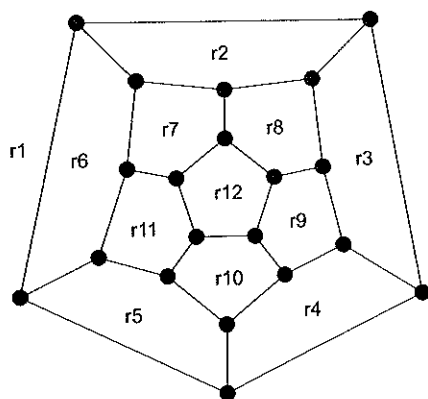


Figure 2.18: A plane graph with 12 regions.

Planar graphs

Let us now take a look at an important class of graphs where topology plays a role, namely graphs that can be drawn in such a way that no edges cross each other, so-called planar graphs.

Definition 2.15: A *plane graph* is a specific embedding of a graph G such that no two edges intersect. If such an embedding exists, G is said to be *planar*.

It is not difficult to see why planarity can play an important role. Consider, for example, designing a transportation network. If the corresponding graph is planar, this means that there is no need for multi-layer crossings such as bridges and tunnels. As another example, consider the design of electrical circuits, such as those for chips. In this case, it is important that the wires that connect components do not cross each other. Unfortunately, designing modern chips under the constraint that the associated graph must be planar is very difficult, if not impossible. The alternative, is to design chips as a collection of layers, each layer having an associated planar graph. We will later discuss another intriguing application of planar graphs, namely the coloring of maps. Before doing so, we first consider a number of characteristic properties of planar graphs.

When considering a plane graph, we will observe a number of regions (also called faces), which are enclosed by the edges of the graph. For example, Figure 2.18 shows a graph with 12 regions. Each region, except r_1 , is enclosed by a cycle. Region r_1 is referred to as an **exterior region**; the others are **interior regions**.

A useful property of planar graphs was formulated by the famous ver-

satile Swiss mathematician Leonhard Euler (1707-1783), who is generally considered as one of the greatest mathematicians ever.

Theorem 2.7 (Euler's formula): For a plane graph G with n vertices, m edges, and r regions, we have that $n - m + r = 2$.

To prove this theorem, we need to consider an important property of an **acyclic graph**, that is, a graph containing no cycles, also known as a **tree**. Formally, we have:

Definition 2.16: A simple, connected graph having no cycles is called a **tree**. A simple graph having only trees as its components, is called a **forest**.

Lemma 2.1: Any tree T with n vertices has $|E(T)| = n - 1$ edges.

Proof. We prove this lemma by induction on the number of vertices. Clearly, when $n = 1$ there can be no edges and the lemma is seen to hold. Now assume the lemma holds for all trees with less than n vertices. Let H be a tree with $n \geq 2$ vertices, and edge $\langle u, v \rangle \in E(H)$. If we remove this edge, then the result will be two separate subgraphs G_1 and G_2 , for otherwise $\langle u, v \rangle$ was part of a cycle. Both subgraphs are acyclic, each with less than n vertices, so that $|E(G_1)| = |V(G_1)| - 1$ and $|E(G_2)| = |V(G_2)| - 1$. Because we have not removed any vertices, we know that

$$|E(H)| = |E(G_1)| + |E(G_2)| + 1 = |V(G_1)| - 1 + |V(G_2)| - 1 + 1 = n - 1$$

which completes the proof. $\square$

Note 2.14 (Proof techniques)

This is the first time we have encountered a **proof by induction**. This type of proof consists of two parts. First, a situation is shown to hold for some initial value n (in our example, the number of vertices for which we first consider one vertex). Then, assuming the situation is valid for $k > 1$, we prove that it also holds for $k + 1$. In doing so, we have then completed the proof.

Proof by induction is extremely important and you should make sure that you not only understand it well, but also that you are proficient in applying it. d'Angelo and West [2000] devote a complete chapter to the principle of induction and provide many examples of its use. Formally, induction is defined by considering the natural numbers, that is, $\mathbb{N} \stackrel{\text{def}}{=} \{1, 2, \dots\}$. We then have the following important theorem.

Theorem 2.8 (Principle of induction): Let $S(n)$ be a mathematical statement formulated in terms a natural number n . $S(n)$ is true if the following two statements are true:

1. $S(1)$ is true
2. for any $k \in \mathbb{N}$, if $S(k)$ is true, then $S(k+1)$ is true

What this theorem tells us, is that to conduct a proof by induction, we need to first show that $S(1)$ holds. Secondly, we need to show that if $S(k)$ is true, then $S(k+1)$ is also true. We can then conclude that $S(n)$ is true for any $n \in \mathbb{N}$. In practice this means that we show $S(1)$ to be true, then *assume* that $S(k)$ is true for $k > 1$, after which we need to show that $S(k+1)$ is true based on that assumption.

Of course, showing that $S(k+1)$ is true is often the nasty part. A common approach is to try to reduce the situation for $k+1$ to $S(k)$. This is exactly what happened in the case of our lemma: we simply removed an edge which lead to subgraphs of smaller size for which we knew that our statement $n = m + 1$ was true. From there on, we could subsequently count the number of vertices and edges in the original graph.

Using this lemma, we can now complete our proof of Euler's formula, again by means of induction:

Proof of Theorem 2.7. The proof is by induction on r , the number of regions. If $r = 1$, then there is only a single region, which means there cannot be a region enclosed by edges of G . In other words, G must be acyclic, in which case $m = n - 1$ and thus $n - m + r = n - (n - 1) + 1 = 2$. For $r = 1$ the formula is therefore seen to be true.

Now assume the formula is true for all plane graphs with less than r regions, and let G be a plane graph with $r > 1$ regions. Choose an edge e (which is not a cut edge) and consider the subgraph $G^* = G - e$. As e was part of a cycle, we will have merged two regions, reducing the total number of regions by 1. In that case, we know that Euler's formula is true, and as a consequence, $|V(G^*)| - |E(G^*)| + (r - 1) = 2$. Considering that $|V(G^*)| = |V(G)|$ and $|E(G^*)| = |E(G)| - 1$, we now obtain $|V(G)| - (|E(G)| - 1) + r - 1 = |V(G)| - |E(G)| + r = 2$, completing our proof. $\square$

Euler's formula is important as it allows us to derive a number of properties by which we can more easily determine whether a given graph is planar or not. To this end, we first prove the following:

Theorem 2.9: For any connected simple planar graph G with $n \geq 3$ vertices and m edges, we have that $m \leq 3n - 6$

Proof. Consider a region f in any plane graph of G . For any interior region, let $B(f)$ denote the number of edges by which f is enclosed, i.e., the length

of its "border." Obviously, $B(f) \geq 3$ for any interior region. However, with $n \geq 3$ we also have that the exterior region is "bounded" by at least 3 edges. Therefore, if there are a total of r regions, then clearly $\sum B(f) \geq 3r$. On the other hand, it is not difficult to see that $\sum B(f)$ counts every edge in G once or twice, and hence $\sum B(f) \leq 2m$, so that we obtain $3r \leq \sum B(f) \leq 2m$, and thus $r \leq \frac{2}{3}m$. From Theorem 2.7 we then derive that $m = n + r - 2 \leq n + \frac{2}{3}m - 2$, so that $m \leq 3n - 6$. $\square$

Note that this theorem gives us a *necessary* condition for a simple graph to be planar. In other words, if we have a simple graph G for which $m > 3n - 6$, then G cannot be planar. It is not a *sufficient* condition, as we will show shortly. Furthermore, what we learn from this theorem is that a planar graph will have relatively few edges, which is intuitively clear. We can use it to prove that the complete graph on 5 vertices, that is, K_5 cannot be planar.

Corollary 2.4: The complete graph on 5 vertices, K_5 is nonplanar.

Proof. With $n = |V(K_5)| = 5$ and $m = |E(K_5)| = \binom{5}{2} = 10$, we have that $m \not\leq 3n - 6$, so that K_5 cannot be planar. $\square$

Note 2.15 (More information)

There are two novelties in this proof. First, we introduced the notation $\binom{n}{k}$, which is pronounced as " n choose k ," and is defined as

$$\binom{n}{k} \stackrel{\text{def}}{=} \frac{n!}{(n-k)!k!}$$

Second, we are stating that the number of edges in K_n is equal to $\binom{n}{2}$. Considering that we have n vertices in K_n , it should be clear that to construct K_n , we need to consider exactly all pairs of vertices. Obviously, there are exactly $\binom{n}{2}$ of such pairs. Another way of counting the number of edges in K_n is as follows. Assume that the vertices are labeled $\{1, 2, \dots, n\}$. For vertex 1, we can choose from $n - 1$ vertices to join it to. After that, there are only $n - 2$ vertices to join to vertex 2 (because vertex 1 is already joined with vertex 2). For vertex 3, we can choose from $n - 3$ vertices, and so. In other words, the total number of edges in K_n is equal to:

$$|E(K_n)| = (n - 1) + (n - 2) + (n - 3) + \dots + 2 + 1 = \frac{1}{2}n(n - 1)$$

To show that $\sum_{i=1}^{n-1} i = \frac{1}{2}n(n - 1)$ is left as an exercise.

Analogous to a complete graph, we also have **complete bipartite graphs** $K_{p,q}$, which is a simple graph consisting of the two disjoint set of vertices V_1

and V_2 as in Definition 2.14 on page 46, with $p = |V_1|$ and $q = |V_2|$, and a total of $p \cdot q$ edges. An observation is now the following:

Theorem 2.10: *The complete bipartite graph $K_{3,3}$ is nonplanar.*

Proof. Because $n = |V(K_{3,3})| = 6$ and $m = |E(K_{3,3})| = 9$, we find that $m \leq 3n - 6$, so that this will not give us evidence that $K_{3,3}$ is indeed nonplanar. Instead, we need to follow a similar reasoning as for the proof of Theorem 2.9. First, note that each interior region f in any $K_{p,q}$ will necessarily be enclosed by an even number of edges. Again, if $B(f)$ denotes the number of edges enclosing interior region f , and realizing that also the exterior region will be “bounded” by at least four edges, we find that $\sum B(f) \geq 4r$, where r is the total number of regions. Because edges are counted twice, we should have that $4r \leq 2m = 18$. However, Euler’s formula tells us that $r = 2 - n + m = 2 - 6 + 9 = 5$, so that $4r \not\leq 18$. Therefore, $K_{3,3}$ cannot be planar. $\square$

Note 2.16 (Mathematical language)

Indeed, as we stated above, the mere fact that $m \leq 3n - 6$ is not enough to conclude that a graph is planar. In other words, it is not a *sufficient* condition.

With these two results, we can now conclude that:

Corollary 2.5: *Any connected, simple graph having a subgraph isomorphic to either K_5 or $K_{3,3}$ cannot be planar.*

EXTENSIONS

